

考点一：计算机的发展

发展阶段	使用器件	运算速度 (次/秒)	主存 (内存)	辅存 (外存)	特 点
第1代 (1946-1957年)	电子管 (真空管)	几千~几万	水银延迟线 磁鼓、磁芯	穿孔卡片 穿孔纸带	使用机器语言编程 无操作系统 Vacuum Tube
第2代 (1958-1964年)	晶体管	几十万~几百万	磁芯	磁鼓、磁带、 磁盘	主要使用汇编语言编程，开始使用FORTRAN、 COBOL等高级语言 单道批处理系统 Transistor
第3代 (1965-1971年)	集成电路	几百万~几千万	半导体存储器	磁带、磁盘	高级语言进一步发展，出现了B语言（C语言 的前身） 多道批处理系统，分时系统 Integrated Circuit
第4代 (1972至今)	超大规模 集成电路	几十亿~几千亿	半导体存储器	磁盘、磁带、 光盘、 半导体存储器	各种高级语言（例如C/C++、Java、Python） 现代操作系统



考点一：计算机的发展

摩尔定律



戈登·摩尔 (Intel创始人之一)

集成电路上可容纳的晶体管数目大约每经过18到24个月便增加一倍。

就是说，处理器的性能大约每两年翻一倍，同时价格下降为先前的一半。

考点一：计算机的发展

软件——编程语言



考点一：计算机的发展

■ 随着硬件和软件的不断发展，人们又创造出了一类程序，称为**操作系统**（属于系统软件）。

☐ 操作系统提供了在**汇编语言和高级语言的使用和实现过程中所需的某些基本操作**。

☐ 操作系统负责**控制并管理计算机系统全部硬件资源**（例如CPU、内存和外部设备）和**软件资源**（例如编译程序、应用程序等）。

☐ 操作系统**为用户使用计算机系统提供了极为方便的条件**。

■ 随着计算机应用领域的逐渐扩大，还相应地出现了**其他各类系统软件**（例如数据库管理系统、网络系统等）以及多种多样应用软件。

■ 随着软件的进一步发展，将会出现**更高级的计算机语言**，其发展方向是标准化、积木化、产品化以及智能化，最终向自然语言发展，它们能够自动生成程序。

考点二：冯诺依曼结构



电子数字积分机和计算机ENIAC (1946年)
(Electronic Numerical Integrator and Computer)

■ 采用**十进制**，**没有内存存储器**。

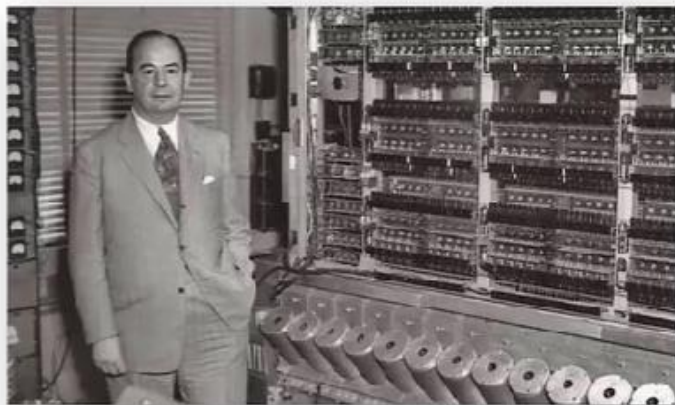
■ 需要通过**旋钮、开关和接插线的不同位置来表示程序**。

- ☐ 在ENIAC上设置一个实用程序，往往需要几天甚至几周的时间。如非必要，使用者很少愿意修改它。
- ☐ 因此，尽管ENIAC是通用的，却总在一段时间内只专用于某个问题（比如弹道计算），它的通用价值被大大削弱。
- ☐ 如果频繁地设置不同程序，机器在很大一部分时间里会处于设置程序过程而无法运行，它的高速性能又被大大浪费了。

考点二：冯诺依曼结构

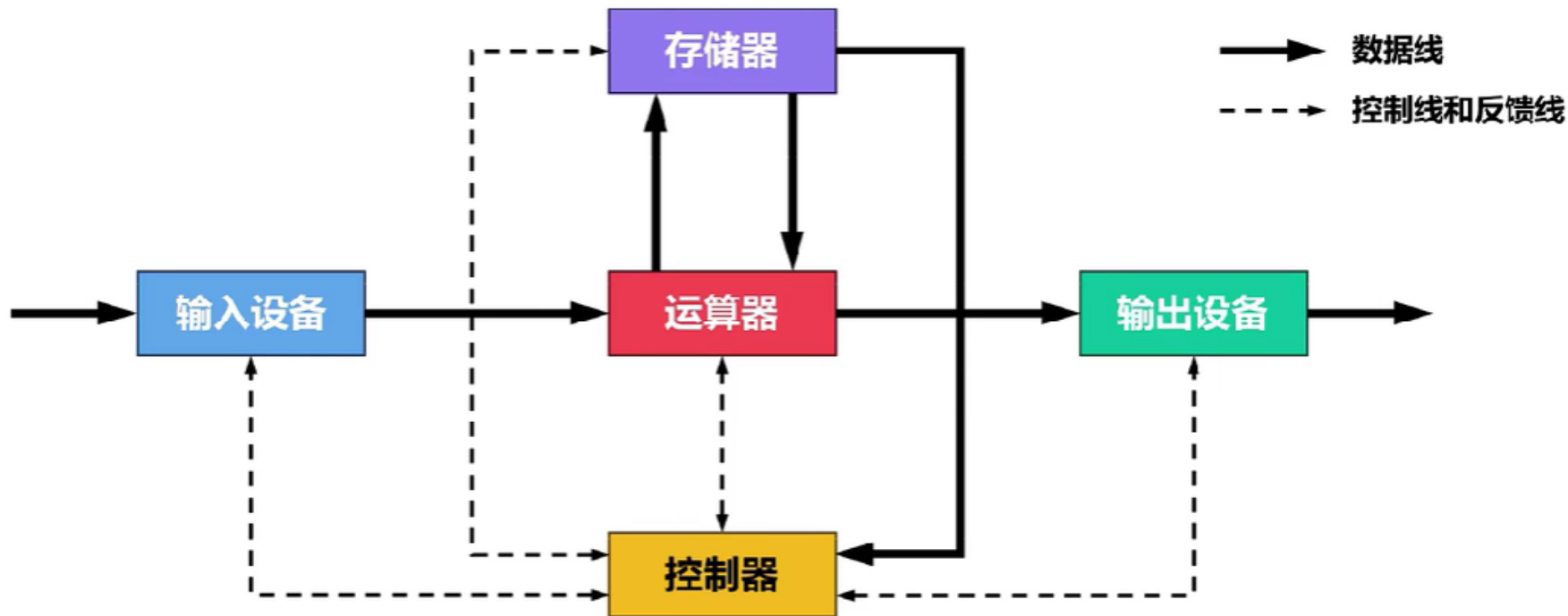
■ 冯·诺依曼计算机的主要特点如下：

- ☐ 构成程序的指令和数据均采用**二进制**表示。
- ☐ **指令和数据存放在存储器**中，按地址访问。
- ☐ **指令在存储器中按顺序存放**。一般情况下，指令是**顺序执行的**。
- ☐ 指令由**操作码**和**地址码**组成：
 - ◇ 操作码用来表示执行何种操作。
 - ◇ 地址码用来表示操作数在存储器中的位置。
- ☐ 机器**以运算器为中心**，输入/输出设备与存储器间的数据传送通过运算器完成。
- ☐ 计算机硬件由**运算器、控制器、存储器、输入设备/输出设备**5大部件组成。



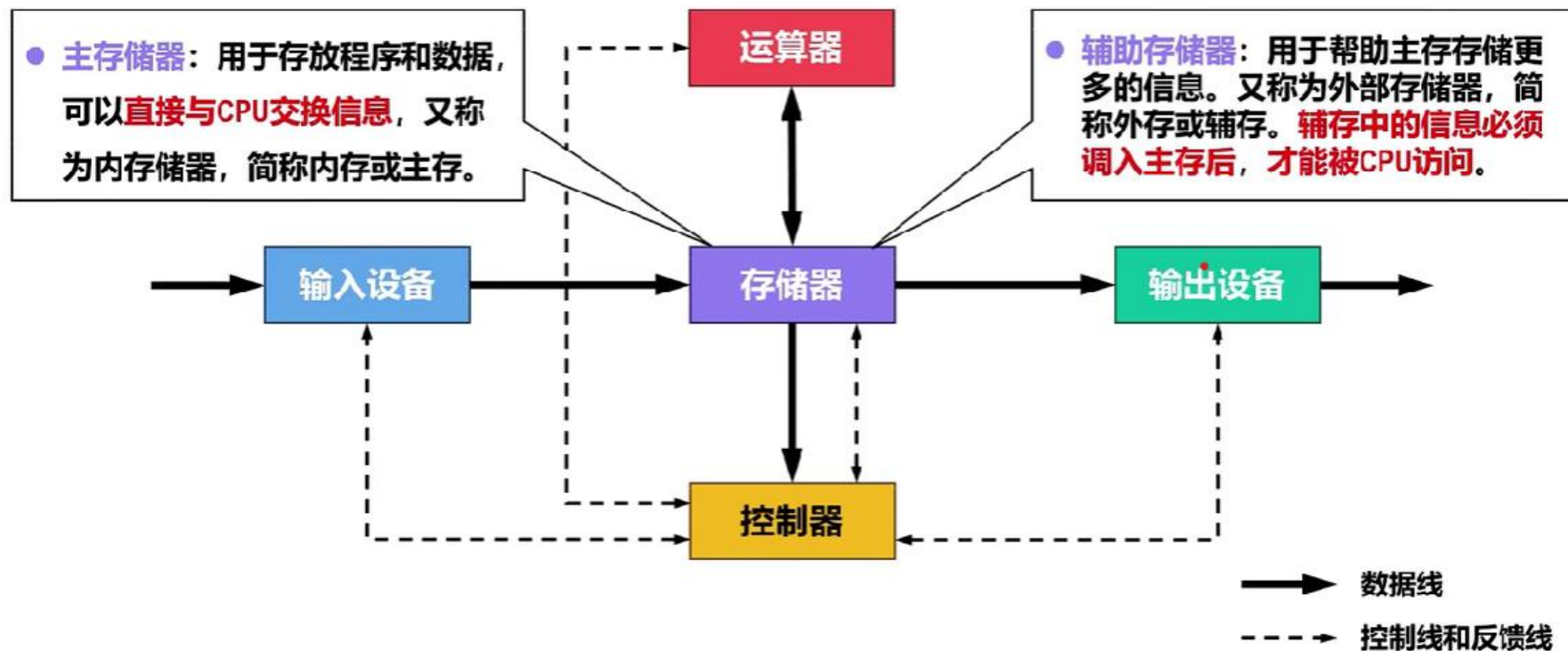
冯·诺依曼与EDVAC机

考点二：冯诺依曼结构



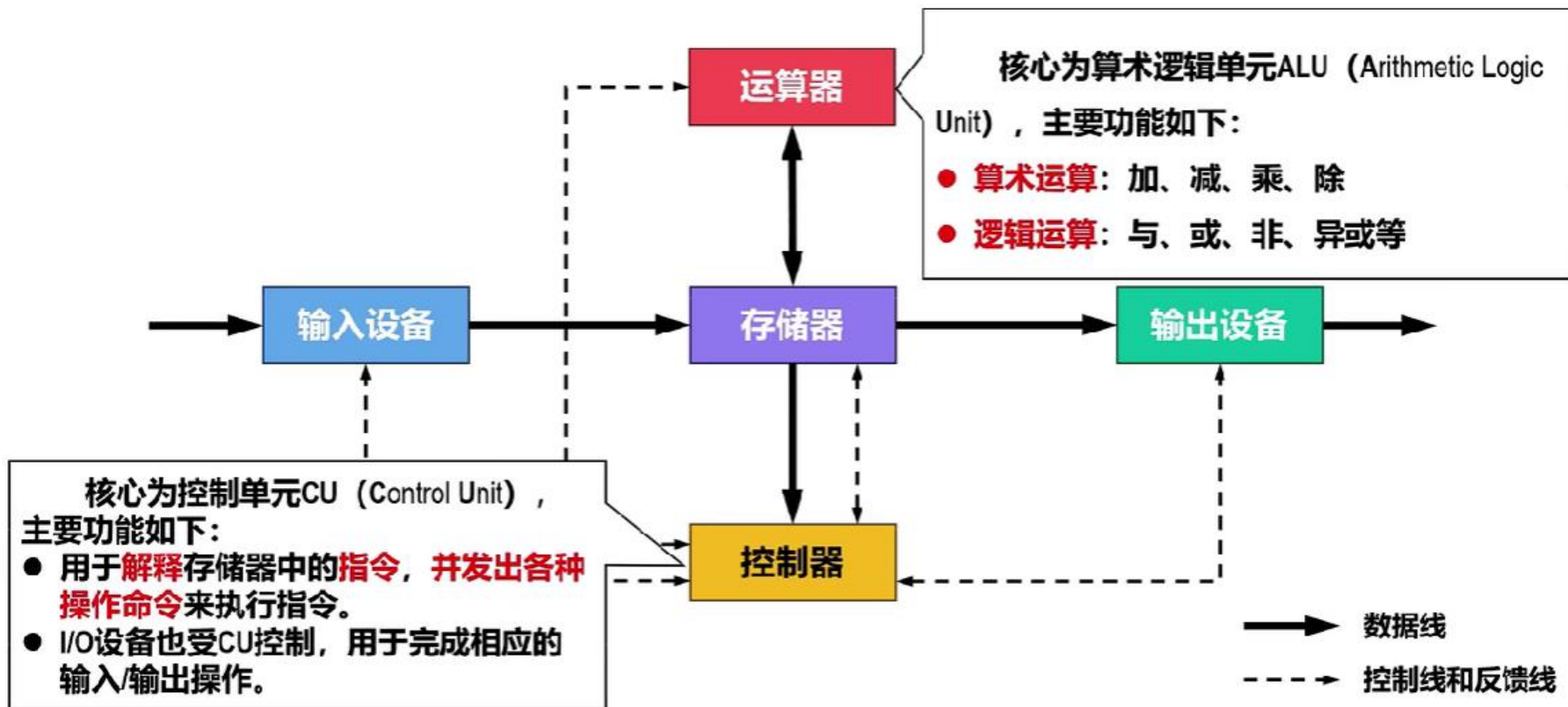
冯诺依曼结构：以运算器为中心，每次输入输出操作均需要运算器参与

考点二：冯诺依曼结构



现代计算机结构：以存储器为中心

考点二：冯诺依曼结构



现代计算机结构：以存储器为中心

考点二：冯诺依曼结构

【例题】下列关于冯·诺依曼结构计算机基本思想的叙述中，错误的是（）

- A.程序的功能都通过中央处理器执行指令实现
- B.指令和数据都用二进制数表示，形式上无差别
- C.指令按地址访问，数据都在指令中直接给出
- D.程序执行前，指令和数据需预先存放在存储器中

◇ 操作码用来表示执行何种操作。

◇ 地址码用来表示操作数在存储器中的位置。

考点二：冯诺依曼结构

- 【例题】以下有关计算机各功能部件的叙述中，错误的是()
- A. 存储器的主要功能是存储程序和数据
 - B. 控制器的主要功能是解释存储器中的指令，并发出各种操作命令来执行指令，I/O设备也受控于控制器
 - C. 输入设备/输出设备的主要功能是完成用户与计算机之间的信息交互
 - D. 运算器的主要功能是进行算术运算

运算器的核心是算术逻辑单元ALU，主要功能如下：

- **算术运算：**加、减、乘、除
- **逻辑运算：**与、或、非、异或、移位等

考点三：计算机性能衡量

硬件与计算机系统性能的关系

- 硬件是构建计算机系统的**物理组件**（例如CPU、内存、外部设备等）。
- 硬件对于计算机系统的性能有着**重要影响**，因为它决定了系统的**计算能力、数据传输速率和存储容量**。
 - CPU的时钟频率决定了CPU每秒钟可以执行的指令数量。
 - 内存带宽会影响数据的读写速率。

软件与计算机系统性能的关系

- 软件包括用于控制、管理、应用计算机系统的各类**系统软件和应用软件**。
- 软件的优化可以**显著影响**计算机系统的性能，因为**合理的算法和代码实现可以更有效地利用硬件资源**。
 - 操作系统的调度算法会影响多任务处理的效率，从而影响系统的响应时间。
 - 在图像处理任务中，优化的软件算法可以减轻CPU和内存的负担，提高图形处理速度。
 - 软件层面的并行计算可以更好地利用多核处理器，提高吞吐量。

综上所述，计算机系统的性能指标涵盖了硬件和软件两个层面，它们之间密切相关。优化硬件可以提供更快速的处理速度，而优化软件可以更有效地利用这些硬件资源，从而共同实现更好的系统性能。

考点三：计算机性能衡量

基本性能指标

机器字长

主存容量

吞吐量

响应时间

■ 机器字长是指CPU一次能够处理的二进制数据的位数，也就是构成二进制数据的比特（bit，简称为b）数量。

☐ 机器字长与CPU内部用于整数运算的ALU的位数以及通用寄存器的宽度相等。

■ 机器字长对计算机系统性能的主要影响如下：

☐ 字长越长，数的表示范围就越大、精度也越高。

☐ 字长越长，计算精度也越高。

☐ 字长还会影响计算速度。

■ 随着硬件技术的发展，计算机的字长已经从早期的8位逐渐增加到16位、32位、64位甚至更高。

考点三：计算机性能衡量

基本性能指标

机器字长

主存容量

吞吐量

响应时间

■ 主存容量是指主存储器（内存）能够存储的最大信息量。

□ 假设主存包含 M 个存储单元，每个存储单元可存储 N 个二进制位，可以通过下式计算主存容量：

$$\text{主存容量} = N \times M \text{ (位或 } b \text{)}$$

数据量的单位	换算关系
比特 (b)	基本单位
字节 (B)	1B = 8bit
千字节 (KB)	KB = 2^{10} B
兆字节 (MB)	MB = K·KB = 2^{20} B
吉字节 (GB)	GB = K·MB = 2^{30} B
太字节 (TB)	TB = K·GB = 2^{40} B

■ 增加主存（内存）容量可以减少程序运行期间对辅存（外存）的访问，由于访问内存的速度远大于访问外存的速度，因此可以提高程序的执行速度，进而提高计算机系统的性能。

考点三：计算机性能衡量

基本性能指标

机器字长

主存容量

吞吐量

响应时间

■ 吞吐量是指计算机系统在**单位时间内能够处理的信息量**。

■ 影响吞吐量的主要因素如下：

- ☐ CPU的处理能力。
- ☐ 内存（主存）的访问速度。
- ☐ 外存（辅存，例如硬盘）的访问速度。

考点三：计算机性能衡量

基本性能指标

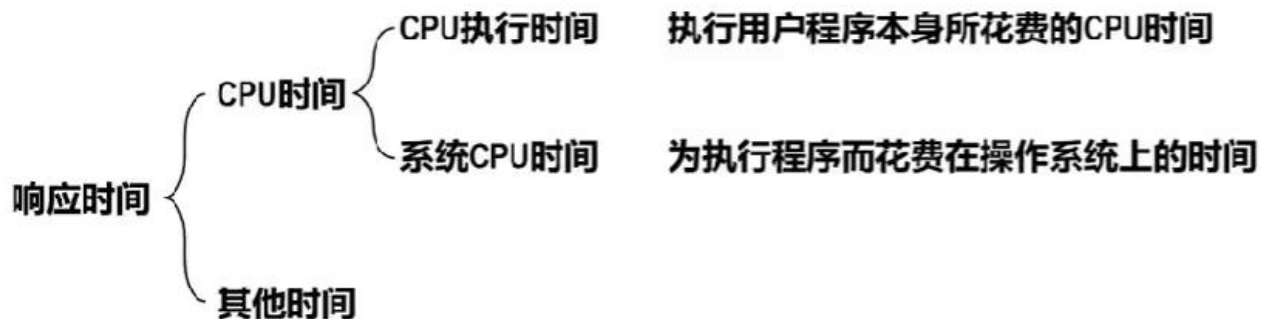
机器字长

主存容量

吞吐量

响应时间

■ 响应时间是指从向计算机系统提交作业开始，到系统完成作业为止所需要的时间。



很难精确区分一个程序执行过程中的CPU执行时间和系统CPU时间，在没有特别说明的情况下，基于CPU执行时间进行计算机性能评价。

考点三：计算机性能衡量

运算相关性能指标

CPU时钟频率和时钟周期

CPI

CPU执行时间

IPC

MIPS

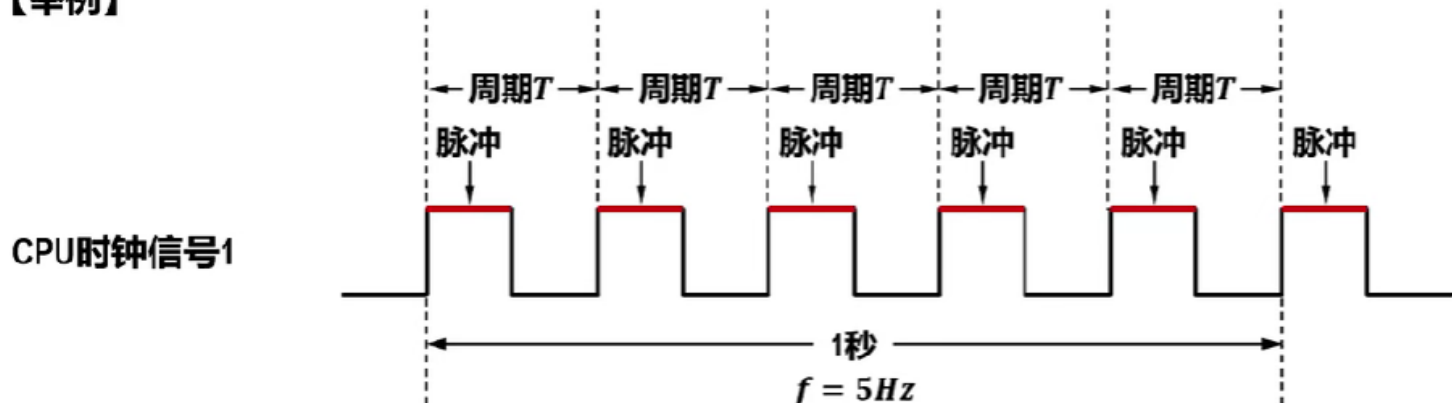
MFLOPS

■ **CPU时钟信号**是一个**基本定时信号**，它是一种固定频率的**脉冲信号**，用于驱动计算机内部各组件协调工作。

□ 这种脉冲信号的频率被称为**CPU时钟频率**（Clock Rate），基本单位为赫兹（Hz）。

□ **CPU时钟周期**（Clock Cycle或Clock Tick，也可简称为clock或tick）是**CPU时钟频率的倒数**，基本单位为秒（s）。

【举例】



对于同类型的计算机，同一指令执行所需的CPU时钟周期的数量是一样的，因此CPU的时钟频率越高，该指令的执行速度就越快。

考点三：计算机性能衡量

运算相关性能指标

CPU时钟频率和时钟周期

CPI

CPU执行时间

IPC

MIPS

MFLOPS

■ CPI (Cycles Per Instruction) 的意思是**执行一条指令所需要的时钟周期数量**。

■ 由于不同指令的功能不同，即使功能相同的指令还可能有不同的寻址方式，因此**每条指令执行所需的时钟周期数量也可能不同**。

□ 针对一条指令、一类指令、一个程序，它们各自CPI的含义如下：

◇ **一条指令的CPI**：该指令执行所需要的时钟周期数量。

◇ **一类指令**（例如算术运算类指令）的CPI：构成该类指令的所有指令执行所需要时钟周期数量的**平均值**。

◇ **一个程序的CPI**：构成该程序的所有指令执行所需要时钟周期数量的**平均值**。

考点三：计算机性能衡量

运算相关性能指标

CPU时钟频率和时钟周期

CPI

CPU执行时间

IPC

MIPS

MFLOPS

- ◇ **一条指令的CPI**：该指令执行所需要的时钟周期数量。
- ◇ **一类指令**（例如算术运算类指令）**的CPI**：构成该类指令的所有指令执行所需要时钟周期数量的**平均值**。
- ◇ **一个程序的CPI**：构成该程序的所有指令执行所需要时钟周期数量的**平均值**。

$CPI = \text{程序执行所需时钟周期数量} \div \text{程序所包含的指令数量}$

- 如果知道某个程序中共有 n 类不同类型的指令、每类指令的CPI（用 CPI_i 表示）、每类指令的数量在程序所包含的指令数量中所占的比例（用 P_i 表示），则该程序的CPI可用下式计算：

$$CPI = \sum_{i=1}^n (CPI_i \times P_i)$$

考点三：计算机性能衡量

运算相关性能指标

CPU时钟频率和时钟周期

CPI

CPU执行时间

IPC

MIPS

MFLOPS

■ CPU执行时间是指**真正用于用户程序的执行时间**，而不包括为执行用户程序而花费在操作系统、访问主存（内存）、访问辅存（外存）、访问外部设备上的时间。

$$\begin{aligned} \text{CPU执行时间} &= \text{程序执行所需时钟周期数量} \times \text{时钟周期} \\ &= \text{程序执行所需时钟周期数量} \div \text{时钟频率} \end{aligned}$$

考点三：计算机性能衡量

运算相关性能指标

CPU时钟频率和时钟周期

CPI

CPU执行时间

IPC

MIPS

MFLOPS

- IPC (Instructions Per Cycle) 的意思是**每个时钟周期能够执行的指令数量**。
- **IPC是CPI的倒数**，它与CPU架构、指令集、指令流水线技术等密切相关。
- 由于指令流水线技术以及多核技术的发展，目前**IPC的值已经可以大于1**。

考点三：计算机性能衡量

运算相关性能指标

CPU时钟频率和时钟周期

CPI

CPU执行时间

IPC

MIPS

MFLOPS

■ MIPS (Million Instructions Per Second) 的意思是**每秒执行多少百万条指令**。


$$\begin{aligned} MIPS &= (\text{程序所包含的指令数量} \div \text{CPU执行时间}) \div 10^6 \\ CPU\text{执行时间} &= (CPI \times \text{程序所包含的指令数量}) \div \text{时钟频率} \\ MIPS &= (\text{时钟频率} \div CPI) \div 10^6 \end{aligned}$$

用MIPS对不同的机器进行性能比较有时是不准确的，主要原因如下。

- 不同机器的指令集不同，并且指令的功能也不同，在某种机器上的某一条指令的功能，可能在另一种机器上需要用多条指令来实现。
- 不同机器的CPI和时钟周期也不同，因而同一条指令在不同机器上所用的时间也不同。

考点三：计算机性能衡量

运算相关性能指标

CPU时钟频率和时钟周期

CPI

CPU执行时间

IPC

MIPS

MFLOPS

■ MFLOPS (Million Floating-Point Operations Per Second) 的意思是**每秒执行多少百万次浮点运算**。

$$MFLOPS = (\text{浮点运算次数} \div \text{测试程序的执行时间}) \div 10^6$$

$$MFLOPS = 10^6 \cdot FLOPS$$

■ 相较于MFLOPS, 衡量每秒执行浮点运算的次数还有以下更大的单位:

☐ GFLOPS (GigaFLOPS) 的意思是每秒执行多少十亿次浮点运算。

$$GFLOPS = 10^9 \cdot FLOPS$$

☐ TFLOPS (TeraFLOPS) 的意思是每秒执行多少万亿次浮点运算。

$$TFLOPS = 10^{12} \cdot FLOPS$$

☐ PFLOPS (PetaFLOPS) 的意思是每秒执行多少千万亿次浮点运算。

$$PFLOPS = 10^{15} \cdot FLOPS$$

☐ EFLOPS (ExaFLOPS) 的意思是每秒执行多少百亿亿次浮点运算。

$$EFLOPS = 10^{18} \cdot FLOPS$$

☐ ZFLOPS (ZettaFLOPS) 的意思是每秒执行多少十万亿亿次浮点运算。

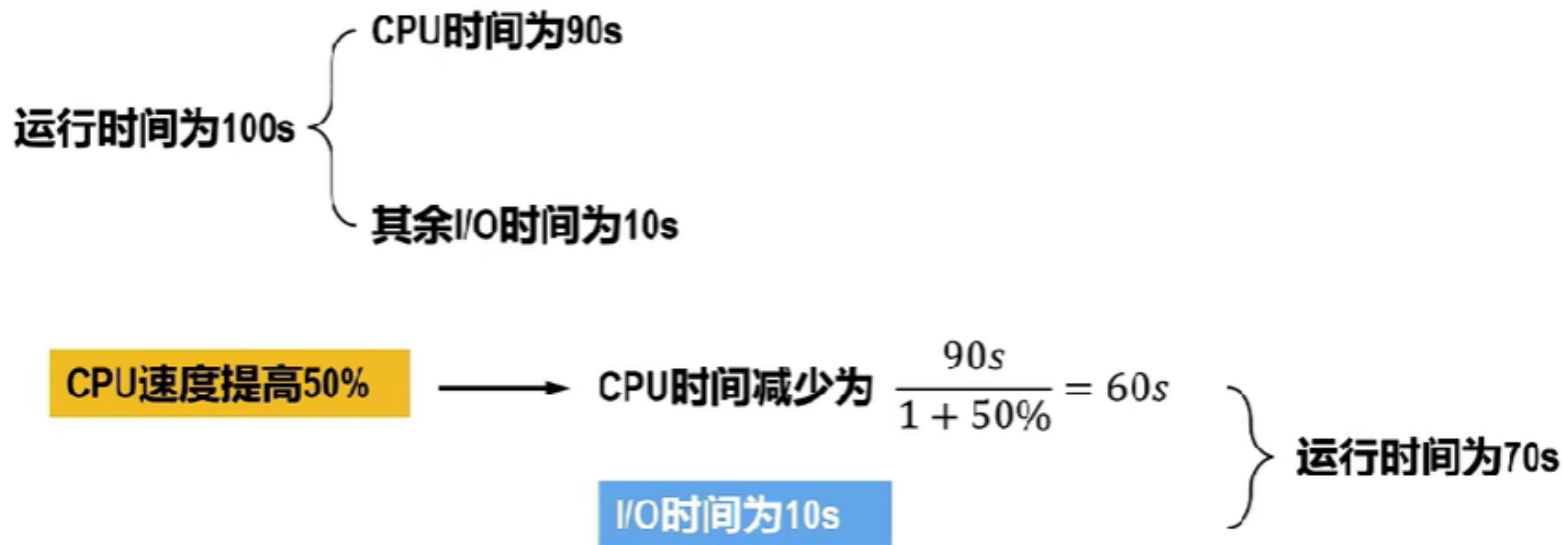
$$ZFLOPS = 10^{21} \cdot FLOPS$$

MFLOPS不能全面反映计算机系统的性能。MFLOPS仅反映浮点运算速度, 其值与所使用的测试程序相关, 不同测试程序中包含的浮点运算量不同, 测试得到的结果也不相同。

考点三：计算机性能衡量

【例题】假定基准程序A在某计算机上的运行时间为100s，其中90s为CPU时间，其余为I/O时间。若CPU速度提高50%，I/O速度不变，则运行基准程序A所耗费的时间是？

解：



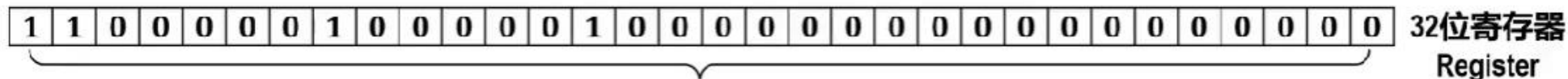
考点三：计算机性能衡量

【例题】假定计算机M1和M2具有相同的指令集体系结构(ISA)，主频分别为1.5GHz和1.2GHz。在M1和M2上运行某基准程序P，平均CPI分别为2和1，则程序P在M1和M2上运行时间的比值是解：


$$\begin{aligned} \text{CPU执行时间} &= \text{程序执行所需时钟周期数量} \times \text{时钟周期} \\ &= \text{程序执行所需时钟周期数量} \div \text{时钟频率} \\ \text{CPI} &= \text{程序执行所需时钟周期数量} \div \text{程序所包含的指令数量} \\ \text{CPU执行时间} &= (\text{CPI} \times \text{程序所包含的指令数量}) \times \text{时钟周期} \\ &= (\text{CPI} \times \text{程序所包含的指令数量}) \div \text{时钟频率} \end{aligned}$$

$$\frac{P \text{在} M1 \text{上的CPU执行时间}}{P \text{在} M2 \text{上的CPU执行时间}} = \frac{CPI_{M1} \times P \text{所包含的指令数量} \div f_{M1}}{CPI_{M2} \times P \text{所包含的指令数量} \div f_{M2}} = \frac{2 \times P \text{所包含的指令数量} \div 1.5}{1 \times P \text{所包含的指令数量} \div 1.2} = 1.6$$

考点四：数据表示



- 某个数值的二进制编码

无符号数
3238264832

有符号数
-1056702464

有符号数
-0.491943359375

有符号数
-8.25
带小数
小数点位置浮动
浮点数

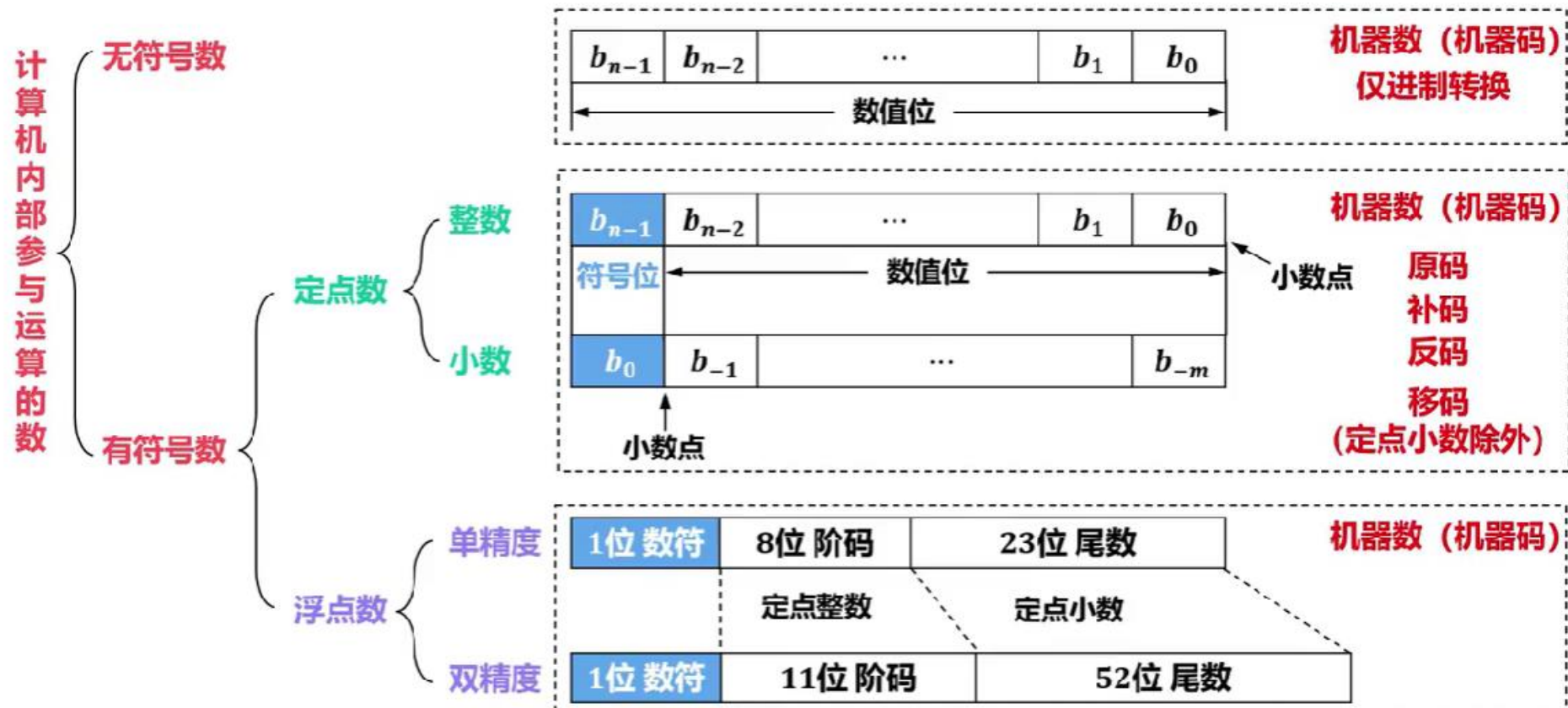
整数

纯小数

小数点位置固定

定点数

考点四：数据表示



考点四：数据表示——原码



定点整数

$$\text{真值 } x = +1011 \quad [x]_{\text{原}} = 0, 1011$$

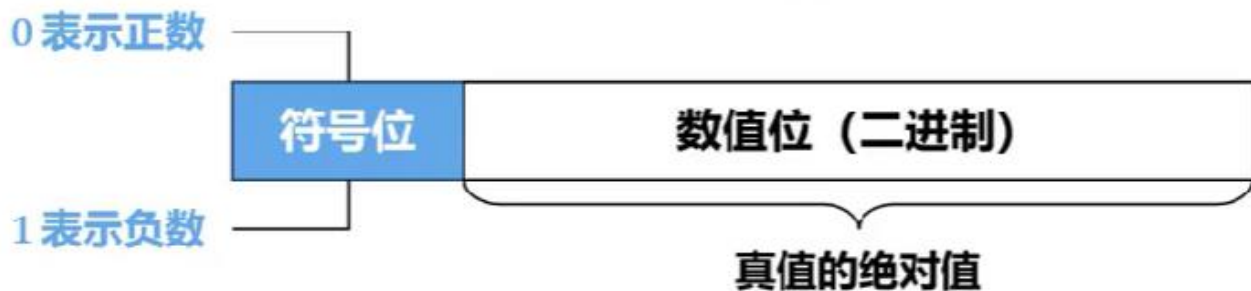
$$\text{真值 } x = -1010 \quad [x]_{\text{原}} = 1, 1010$$

定点小数

$$\text{真值 } x = +0.1111 \quad [x]_{\text{原}} = 0.1111$$

$$\text{真值 } x = -0.0101 \quad [x]_{\text{原}} = 1.0101$$

考点四：数据表示——原码



原码表示又称为带符号的绝对值表示

定点整数的原码定义

假设真值 x 为定点整数， n 为 x 的原码表示中数值位的位数（比特数量）。

$$[x]_{\text{原}} = \begin{cases} 0, x & 0 \leq x < 2^n \\ 2^n + |x| & -2^n < x \leq 0 \end{cases}$$

定点小数的原码定义

假设真值 x 为定点小数

$$[x]_{\text{原}} = \begin{cases} x & 0 \leq x < 1 \\ 1 + |x| & -1 < x \leq 0 \end{cases}$$

考点四：数据表示——原码

定点整数的原码定义

假设真值 x 为定点整数， n 为 x 的原码表示中数值位的位数（比特数量）。

$$[x]_{\text{原}} = \begin{cases} 0, x & 0 \leq x < 2^n \\ 2^n + |x| & -2^n < x \leq 0 \end{cases}$$

定点小数的原码定义

假设真值 x 为定点小数

$$[x]_{\text{原}} = \begin{cases} x & 0 \leq x < 1 \\ 1 + |x| & -1 < x \leq 0 \end{cases}$$

【练习1】将真值0表示为原码

0.0000/1.0000

考点四：数据表示——原码

定点整数的原码定义

假设真值 x 为定点整数， n 为 x 的原码表示中数值位的位数（比特数量）。

$$[x]_{\text{原}} = \begin{cases} 0, x & 0 \leq x < 2^n \\ 2^n + |x| & -2^n < x \leq 0 \end{cases}$$

定点小数的原码定义

假设真值 x 为定点小数

$$[x]_{\text{原}} = \begin{cases} x & 0 \leq x < 1 \\ 1 + |x| & -1 < x \leq 0 \end{cases}$$

【练习1】计算：+5-3

原 码							
	0	0	0	0	1	0	1
	1	0	0	0	0	1	1
				1	1	1	
负数	1	0	0	0	1	0	0

考点四：数据表示——补码 2^n 's complement

模的概念

- ☐ **模**（或称模数）是一个数值计量系统的**计量范围**，记作mod或M。
- ☐ 只要确定了“模”，就可找到一个**与负数等价的正数**来代替此负数，该正数就是负数的**补数**。
- ☐ **超过计量范围的数都应该自动舍弃模数**。

最高位 b_7 的进位 b_8 ，
舍弃该位，该位的
位权值 2^8 就是模数
超过计量范围
自动舍弃模数

【举例】

b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0	8 位寄存器 (mod 2^8)
0	0	0	0	0	0	0	0	最小值 0
0	0	0	0	0	0	0	1	
0	0	0	0	0	0	1	0	
0	0	0	0	0	0	1	1	
⋮								
1	1	1	1	1	1	1	1	最大值 255
0	0	0	0	0	0	0	0	256

1

考点四：数据表示——补码

■ 将补数的概念应用到计算机内部，便出现了补码这种机器码（机器数）。

☐ 正数的补码：符号位为0，数值位就是它本身。

☐ 负数的补码：等于模数加上该负数本身，而模数就是最高位进位的位权值。

定点整数的补码定义

假设真值 x 为定点整数， n 为 x 的补码表示中数值位的位数（比特数量），加上1个符号位， x 的补码表示共有 $n+1$ 位，最低位的位权值为 2^0 ，而最高位（符号位）的位权值为 2^n ，因此最高位进位的位权值为 2^{n+1} ，即模数为 2^{n+1} 。

$$[x]_{\text{补}} = \begin{cases} 0, x & 0 \leq x < 2^n \\ 2^{n+1} + x & -2^n \leq x < 0 \quad (\text{mod } 2^{n+1}) \end{cases}$$

【举例】

某计算机的字长为8位，真值 x 为定点整数 $(-10101)_2$ ，求 $[x]_{\text{补}}$ 。

模数为 2^8

$$\begin{aligned} [x]_{\text{补}} &= 2^8 + (-10101) \\ &= 100000000 - 10101 \\ &= 11101011 \end{aligned}$$

考点四：数据表示——补码

计算机的字长为8位

原 码	真 值	补 码	真 值
0,0000000	+ 0	0,0000000	+ 0
0,0000001	+ 1	0,0000001	+ 1
0,0000010	+ 2	0,0000010	+ 2
0,0000011	+ 3	0,0000011	+ 3
⋮	⋮	⋮	⋮
0,1111110	+ 126	0,1111110	+ 126
0,1111111	+ 127	0,1111111	+ 127
1,0000000	- 0	1,0000000	- 128
1,0000001	- 1	1,0000001	- 127
1,0000010	- 2	1,0000010	- 126
1,0000011	- 3	1,0000011	- 125
⋮	⋮	⋮	⋮
1,1111111	- 127	1,1111111	- 1

考点四：数据表示——补码

■ 将补数的概念应用到计算机内部，便出现了补码这种机器码（机器数）。

☐ 正数的补码：符号位为0，数值位就是它本身。

☐ 负数的补码：等于模数加上该负数本身，而模数就是最高位进位的位权值。

定点小数的补码定义

假设真值 x 为定点小数（纯小数），小数点左侧的位为最高位（符号位），其位权值为 2^0 ，而最高位进位的位权值为 2^1 ，即模数为 $2^1 = 2$ 。

【举例】

$$x = +0.0101 \quad [x]_{\text{补}} = 0.0101$$

$$\begin{aligned} x = -0.0101 \quad [x]_{\text{补}} &= 2 + (-0.0101) \\ &= 10.0000 - 0.0101 \\ &= 1.1011 \end{aligned}$$

$$[x]_{\text{补}} = \begin{cases} x & 0 \leq x < 1 \\ 2 + x & -1 \leq x < 0 \quad (\text{mod } 2) \end{cases}$$

考点四：数据表示——补码

优点

- 补码表示方法使得减法运算可以转换成加法运算。
- 真值0在补码中只有一种表示，这使得补码比原码多表示一个最小负数。
- 符号位可以直接参与运算，运算时符号位的进位作为模会被自动舍弃。

目前计算机中普遍采用补码表示有符号定点整数，例如C语言中char、short、int、long型整数都是采用补码进行表示的。

缺点

- 补码的表示相对原码更加复杂。
 - 原码的数值位与真值的绝对值相同。因此，通过原码可以很容易地得出真值。但是，补码就没有这么简单了。

考点四：数据表示——反码 1's complement

如何求得负数原码的补码？

真值	原码			补码
-1	1,001	符号位不变，数值位取反	1,110	末位加1 1,111
-2	1,010	符号位不变，数值位取反	1,101	末位加1 1,110
-3	1,011	符号位不变，数值位取反	1,100	末位加1 1,101
-4	1,100	符号位不变，数值位取反	1,011	末位加1 1,100
-5	1,101	符号位不变，数值位取反	1,010	末位加1 1,011
-6	1,110	符号位不变，数值位取反	1,001	末位加1 1,010
-7	1,111	符号位不变，数值位取反	1,000	末位加1 1,001

考点四：数据表示——反码 1's complement

■ 反码通常用来作为由原码求补码或者由补码求原码的中间过渡。

☐ 正数的反码：符号位为0，数值位就是它本身。

☐ 负数的反码：符号位为1，数值位就是真值数值位取反。

定点整数的反码定义

假设真值 x 为定点整数， n 为 x 的反码表示中数值位的位数（比特数量）。

$$[x]_{\text{反}} = \begin{cases} 0, x & 0 \leq x < 2^n \\ (2^{n+1} - 1) + x & -2^n < x \leq 0 \end{cases}$$

【举例】

$$x = +1101 \quad [x]_{\text{反}} = 0, 1101$$

$$\begin{aligned} x = -1111 \quad [x]_{\text{反}} &= 2^{4+1} - 1 + (-1111) \\ &= 2^5 - 1 - 1111 \\ &= 10000 - 1 - 1111 \\ &= 1,0000 \end{aligned}$$

考点四：数据表示——反码 1's complement

■ 反码通常用来作为由原码求补码或者由补码求原码的**中间过渡**。

☐ 正数的反码：符号位为0，数值位就是它本身。

☐ 负数的反码：符号位为1，数值位就是真值数值位取反。

定点小数的反码定义

假设真值 x 为定点小数， n 为 x 的反码表示中数值位的位数（比特数量）。

$$[x]_{\text{反}} = \begin{cases} x & 0 \leq x < 1 \\ (2 - 2^{-n}) + x & -1 < x \leq 0 \end{cases}$$

【举例】

$$x = +0.1101 \quad [x]_{\text{反}} = 0.1101$$

$$\begin{aligned} x = -0.1111 \quad [x]_{\text{反}} &= 2 - 2^{-4} + (-0.1111) \\ &= 10.0000 - 0.0001 - 0.1111 \\ &= 1.0000 \end{aligned}$$

考点四：数据表示——反码 1's complement

■ 反码通常用来作为由原码求补码或者由补码求原码的**中间过渡**。

☐ 正数的反码：符号位为0，数值位就是它本身。

☐ 负数的反码：符号位为1，数值位就是真值数值位取反。

定点小数的反码定义

假设真值 x 为定点小数， n 为 x 的反码表示中数值位的位数（比特数量）。

$$[x]_{\text{反}} = \begin{cases} x & 0 \leq x < 1 \\ (2 - 2^{-n}) + x & -1 < x \leq 0 \end{cases}$$

【练习】

$$x = +0.0000 \quad [x]_{\text{反}} = 0.0000$$

$$\begin{aligned} x = -0.0000 \quad [x]_{\text{反}} &= 2 - 2^{-4} + (-0.0000) \\ &= 10.0000 - 0.0001 - 0.0000 \\ &= 1.1111 \end{aligned}$$

真值0在反码中有两种不同的表示

考点四：数据表示——反码 1's complement

反码通常用来作为由原码求补码或者由补码求原码的中间过渡。

☐ **正数的反码：符号位为0，数值位就是它本身。**

☐ **负数的反码：符号位为1，数值位就是真值数值位取反。**

【例】使用反码计算： $+9-5$

真 值		反 码							
+9		0	0	0	0	1	0	0	1
-5		1	1	1	1	1	0	1	0
加法	1	1	1	1	1				
+4	+3	0	0	0	0	0	0	1	1
不相等									

考点四：数据表示——反码 1's complement

■ 反码通常用来作为由原码求补码或者由补码求原码的中间过渡。

☐ 正数的反码：符号位为0，数值位就是它本身。

☐ 负数的反码：符号位为1，数值位就是真值数值位取反。

【例】使用反码计算：+9-5

真 值		反 码							
+9		0	0	0	0	1	0	0	1
-5		1	1	1	1	1	0	1	0
加法		1	1	1	1	1			
+4		0	0	0	0	0	0	1	1
		+						1	
		+4	0	0	0	0	1	0	0

相等

考点四：数据表示——反码 1's complement

优点

- 符号位可以参与运算。

缺点

- 最高位（符号位）产生的进位要加到运算结果的低位（循环进位）
- 真值0在反码中有两种不同的表示

反码在计算机中目前很少被使用

考点四：数据表示——移码

原 码

优点

- 表示方法简单直观

缺点

- 符号位不能直接参与运算
- 真值0在原码中有两种不同的表示

原码在计算机中目前仅仅用于表示浮点数的尾码。

补 码

优点

- 补码表示方法使得减法运算可以转换成加法运算。
- 真值0在补码中只有一种表示，这使得补码比原码和反码多表示一个最小负数。
- 符号位可以直接参与运算，运算时符号位的进位作为模会被自动舍弃。

缺点

- 补码的表示比原码复杂。
 - 原码的数值位与真值的绝对值相同。通过原码可以很容易地得出真值。但是，补码就没有这么简单了。

通常不会涉及定点小数的补码表示。

反 码

优点

- 符号位可以参与运算。

缺点

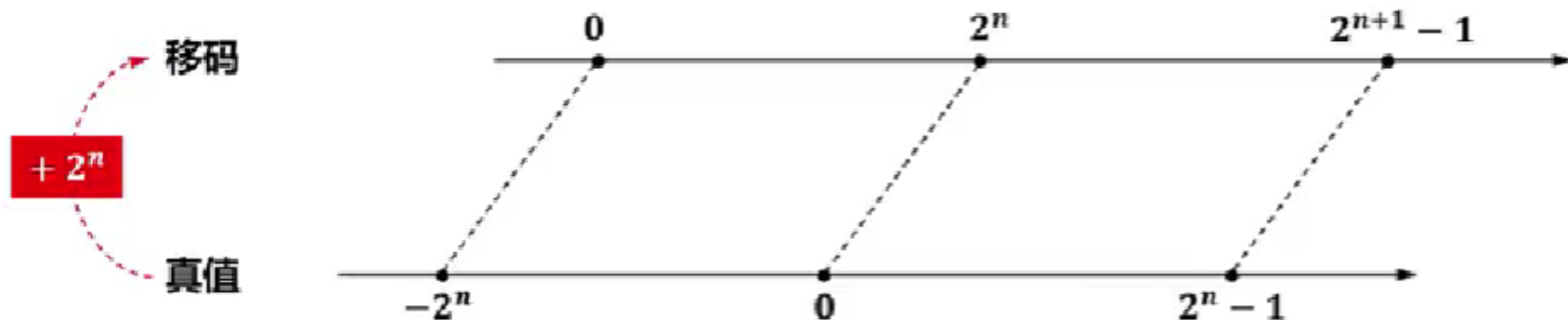
- 最高位（符号位）产生的进位要加到运算结果的低位（循环进位）
- 真值0在反码中有两种不同的表示

反码在计算机中目前很少被使用。通常用来作为由原码求补码或者由补码求原码的中间过渡。

考点四：数据表示——移码

■ 移码就是在真值上加一个常数 2^n 。

- ☐ 在数轴上，移码所表示的范围对应于真值在数轴上的范围向轴的正方向移动 2^n 个单元。
- ☐ 移码只用于定点整数的表示。



考点四：数据表示——移码

移码定义

假设真值 x 为定点整数， n 为 x 的移码表示中数值位的位数（比特数量）。

$$[x]_{\text{移}} = x + 2^n \quad -2^n \leq x < 2^n$$

【举例】

$$x = +1010110 \quad [x]_{\text{移}} = +1010110 + 2^7 = +1010110 + 10000000 = 1,1010110$$

$$x = -1010110 \quad [x]_{\text{移}} = -1010110 + 2^7 = -1010110 + 10000000 = 0,0101010$$

考点四：数据表示——移码

优点

- 真值0在移码中只有一种表示。
- 移码保持了真值原有的大小顺序，可以直接比较大小。
- 最小真值的移码为全0，最大真值的移码为全1，符合人们的习惯。

当浮点数的阶码用移码来表示时，就能很方便地比较阶码的大小。

考点四：数据表示——定点数转换

真值 x 为正数



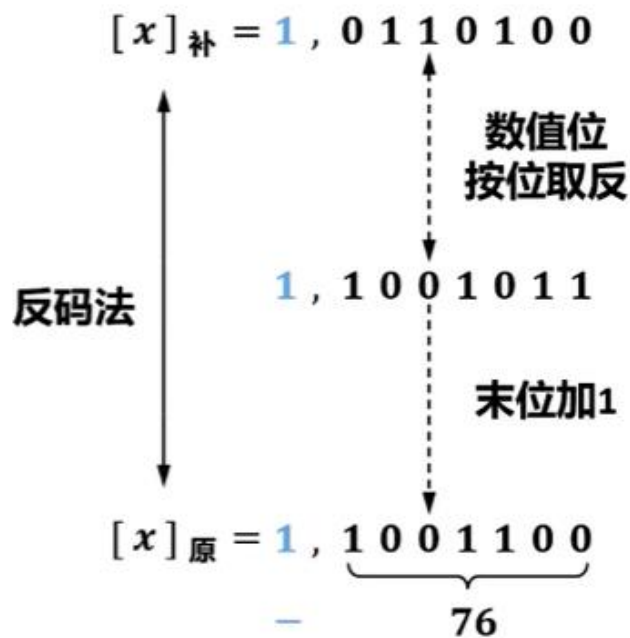
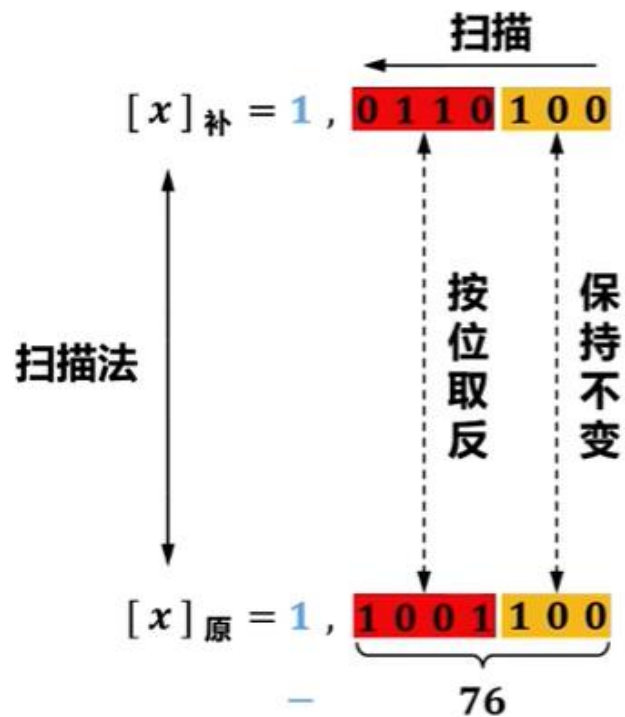
真值 x 为负数



考点四：数据表示——定点数例题

【例】 x 的补码是1, 0110100, 求 x ?

补码的补码是原码



考点四：数据表示——定点数例题

【例】设机器数采用补码形式，若寄存器内容为9BH，则对应的十进制数？

$$[x]_{\text{补}} = 1,001\,1011$$

定点整数的补码定义

假设真值 x 为定点整数， n 为 x 的补码表示中数值位的位数（比特数量），加上1个符号位， x 的补码表示共有 $n+1$ 位，最低位的位权值为 2^0 ，而最高位（符号位）的位权值为 2^n ，因此最高位进位的位权值为 2^{n+1} ，即模数为 2^{n+1} 。

$$[x]_{\text{补}} = \begin{cases} 0, x & 0 \leq x < 2^n \\ 2^{n+1} + x & -2^n \leq x < 0 \quad (\text{mod } 2^{n+1}) \end{cases}$$

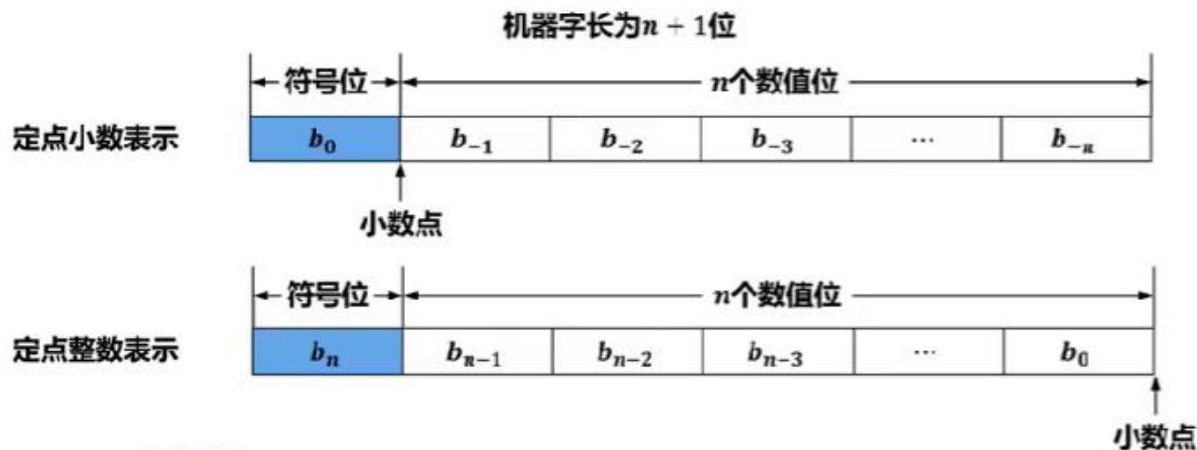
$$10011011 = 2^{7+1} + x$$

$$155 = 256 + x$$

$$x = -101$$

考点四：数据表示——浮点数表示

■ 实际上，计算机中处理的数不一定是纯小数或纯整数（例如圆周率3.1415926）。因此，这样的数不能直接用定点小数或定点整数表示。



$$0.31415926 \times 10^1$$
$$31415926 \times 10^{-7}$$

定点数 比例因子



3.1415926可以用定点小数或定点整数直接表示吗？
显然不能！

希望将浮点数直接表示在寄存器中：

- 小数点的位置由寄存器中指定数位的内容给出；
- 正负号由寄存器中指定数位的内容给出。

考点四：数据表示——浮点数表示

■ 二进制浮点数可采用类似十进制科学记数法的表示方法。

【举例】

十进制浮点数

$$\begin{aligned}408.32 &= 4.0832 \times 10^2 && \text{小数点向右移动2位} \\&= 4083.2 \times 10^{-1} && \text{小数点向左移动1位} \\&= 0.40832 \times 10^3 && \text{小数点向右移动3位}\end{aligned}$$

二进制浮点数

$$\begin{aligned}110.0101 &= 1.100101 \times 2^{10} && \text{小数点向右移动2位} \\&= 1100.101 \times 2^{-1} && \text{小数点向左移动1位} \\&= 0.1100101 \times 2^{11} && \text{小数点向右移动3位}\end{aligned}$$

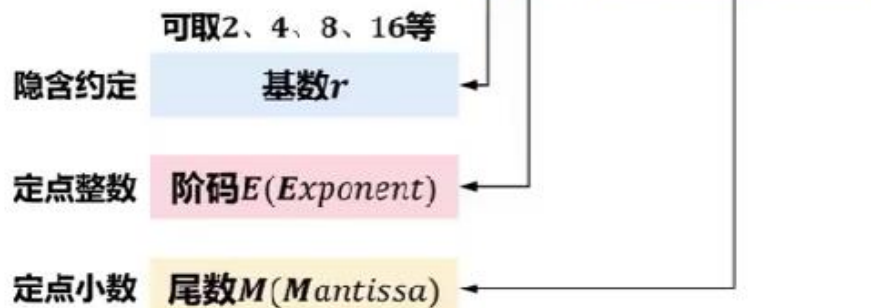
考点四：数据表示——浮点数表示

【举例】

二进制浮点数 $110.0101 = 2^{10} \times 1.100101$

$$= 2^{-1} \times 1100.101$$

$$= 2^{11} \times 0.1100101$$



任意一个二进制数 N 可表示成如下形式：

$$N = r^E \times M$$

浮点数在机器中的表示形式

阶码E						尾数M					
E_f	E_1	E_2	E_3	$\cdots$	E_k	M_f	M_1	M_2	M_3	$\cdots$	M_n
阶符	阶码的数值部分					数符	尾数的数值部分				
- 阶码的位数决定了数据表示的范围，位数越多，能表示的数据范围就越大。 - 阶码的值决定了小数点的位置。						尾数的位数决定了数据的表示精度。阶码长度相同时，分配给尾数的位数越多，数据表示的精度就越高。					

- 阶码可采用原码、补码、反码、移码进行表示。
- 尾数可采用原码、补码、反码进行表示。
- 对应的浮点数表示范围会略有不同。

考点四：数据表示——浮点数表示范围

浮点数在机器中的表示形式

$N = r^E \times M$ (r 可取2、4、8、16等)

正数 N 最大值

正数 N 最小值 (浮点数的最小精度)

负数 N 最小值

负数 N 最大值

【举例】

正数 N 最大值 $2^{+3} \times (+0.75)$

正数 N 最小值 $2^{-3} \times (+0.25)$

负数 N 最小值 $2^{+3} \times (-0.75)$

阶码E						尾数M					
E_f	E_1	E_2	E_3	...	E_k	M_f	M_1	M_2	M_3	...	M_n
阶符	阶码的数值部分					数符	尾数的数值部分				
最大值						正数最大值					
最小值						正数最小值					
最大值						负数最小值					
最小值						负数最大值					
3位阶码 (原码表示) (1个阶符, 2个数值位)						3位尾数 (原码表示) (1个数符, 2个数值位)					
最大值0, 11, 即+3						正数最大值0. 11, 即+0.75					
最小值1, 11, 即-3						正数最小值0. 01, 即+0.25					
最大值0, 11, 即+3						负数最小值1. 11, 即-0.75					
最小值1, 11, 即-3						负数最大值1. 01, 即-0.25					

考点四：数据表示——浮点数表示范围

■ 尽管浮点数有效扩大了数据表示范围，但受机器字长限制，浮点数仍然存在溢出现象。

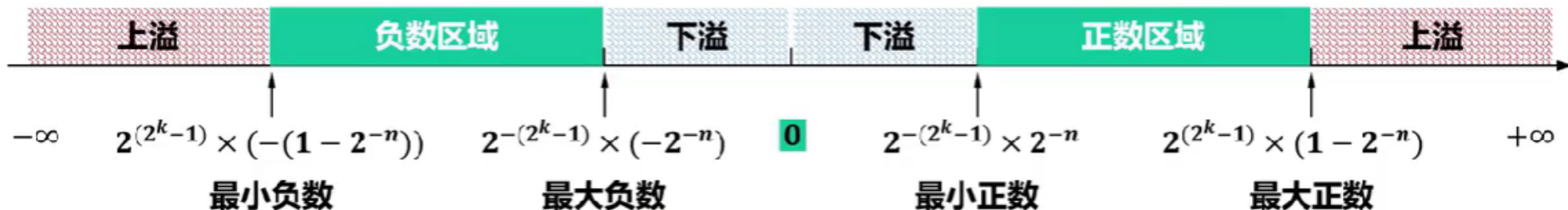
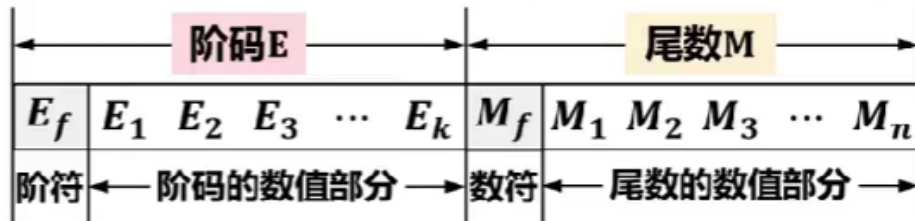
□ 当浮点数的阶码大于最大阶码时，称为上溢，此时机器停止运算，浮点运算器件会显示溢出标志。

□ 当浮点数的阶码小于最小阶码时，称为下溢，虽然此时数据不能被精确表示，但由于发生下溢时数据的绝对值很小，通常将尾数各位强置为0，按机器0处理，此时机器可以继续运行。

■ 当一个浮点数在正、负数区域中但并不在某个数轴刻度上时，也会出现精度溢出的问题，此时只能用近似数表示。

浮点数在机器中的表示形式

$$N = r^E \times M \quad (r \text{ 可取 } 2, 4, 8, 16 \text{ 等})$$

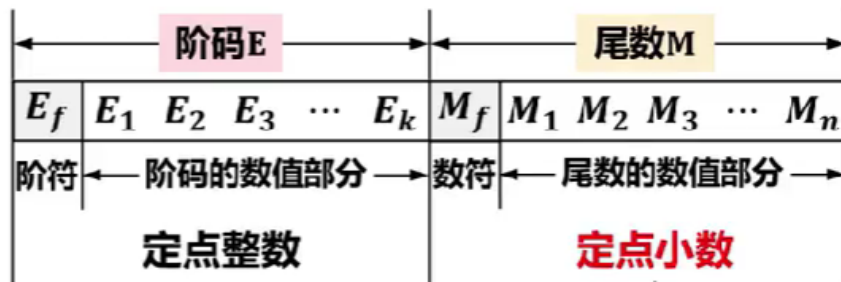


考点四：数据表示——浮点数规格化

浮点数的规格化

浮点数在机器中的表示形式

$$N = r^E \times M \quad (r \text{ 可取 } 2, 4, 8, 16 \text{ 等})$$



带来问题

【举例】

二进制浮点数

110.0101

= 2

11

×

0.1100101

将0.1100101的小数点右移3位

= 2

100

×

0.01100101

将0.01100101的小数点右移4位

阶码E

尾数M

同一浮点数可能存在多种表示形式，也就是会有不同的阶码和尾数的组合。

考点四：数据表示——浮点数规格化

■ 通常要求浮点数在数据表示时对尾数进行规格化处理，即使得尾数的最高数值位必须是一个有效值。

■ 浮点数规格化带来以下好处：

☐ 使浮点数的表示形式唯一。

☐ 使浮点数的表示精度最高。

二进制浮点数

110.0101

= 2

11

×

0.1100101

规格化

= 2

100

×

0.01100101

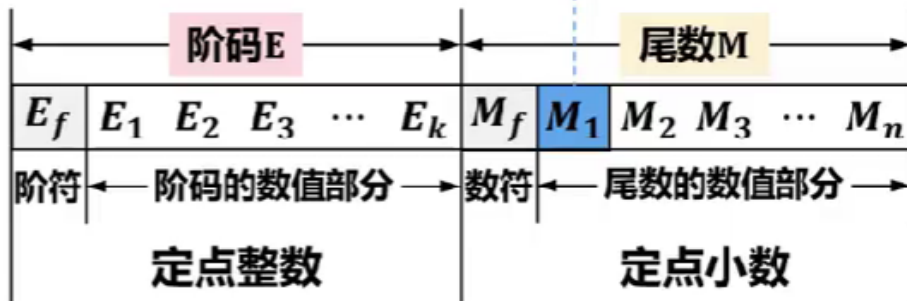
阶码E

尾数M

尾数的数值部分的最高位为1

浮点数在机器中的表示形式

$$N = r^E \times M \quad (r \text{ 可取 } 2, 4, 8, 16 \text{ 等})$$



考点四：数据表示——浮点数规格化

■ 对于非规格化尾数，需要对其进行**规格化操作**，即根据具体形式通过将非规格化**尾数的数值部分**进行**左移或右移**，并**相应减少或增加阶码值**的操作进行规格化，对应的规格化方法分别称为向左规格化（简称**左规**）和向右规格化（简称**右规**）。

■ 对于**基数 r 不同的浮点数**，因其规格化数的形式不同，规格化过程也不同。

□ 当 $r = 2$ 时，尾数**数值部分最高位为1**的数为规格化数。

◇ 左规：尾数数值部分**每左移1位**，**阶码减1**。

【举例】

二进制浮点数 110.0101

非规格化表示

$$2^{100} \times 0.01100101$$

↓ 减1 ↓ 左移1位

规格化表示

$$2^{11} \times 0.11001010$$

$$2^{101} \times 0.001100101$$

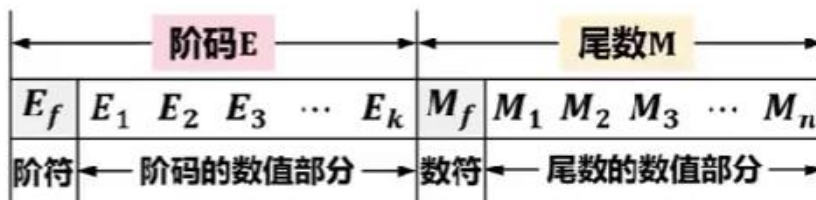
↓ 减2 ↓ 左移2位

$$2^{11} \times 0.110010100$$

考点四：数据表示——规格化浮点数的范围

浮点数在机器中的表示形式

$$N = r^E \times M \quad (r \text{ 可取 } 2, 4, 8, 16 \text{ 等})$$



非规格化



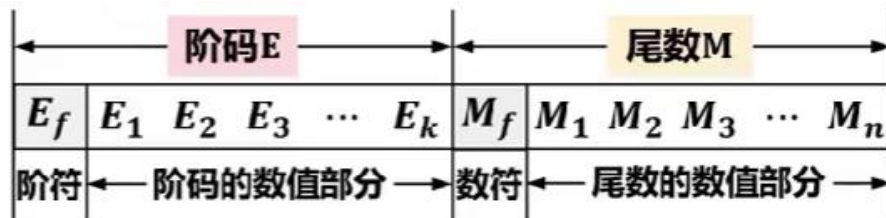
规格化

规格化后，尾数的数值位的最高位为1，尾数的其余位为0

考点四：数据表示——IEEE 754

浮点数在机器中的表示形式

$$N = r^E \times M \quad (r \text{可取} 2, 4, 8, 16 \text{等})$$



- 浮点数的上述表示形式，既没有规定阶码和尾数的位数，也没有规定阶码和尾数采用的机器码形式（原码、反码、补码和移码）。
- 实际上，直到20世纪80年代初，浮点数表示形式还没有统一标准，不同厂商计算机内部浮点数表示形式可能不同。
 - 不同体系结构的计算机之间进行数据传送或程序移植时，必须进行数据格式的转换，并且数据格式转换还会带来运算结果的不一致。
- 因此，美国电气及电子工程师协会（Institute of Electrical and Electronics Engineers, IEEE）于1985年发布了浮点数标准IEEE 754。
 - 目前，几乎所有计算机都采用IEEE 754标准表示浮点数。

考点四：数据表示——IEEE 754

IEEE 754 标准主要包括两种基本的浮点数格式：

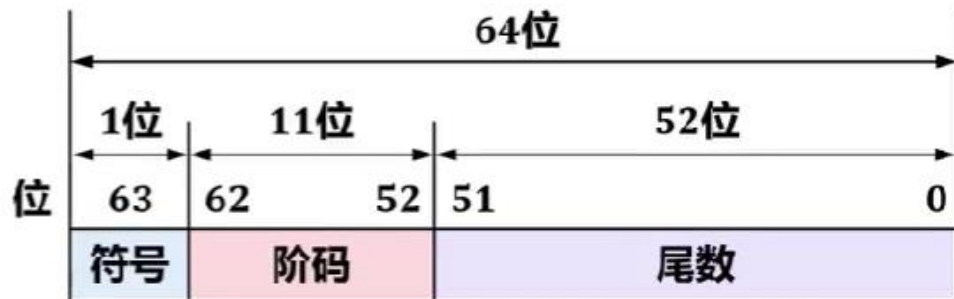
□ 32位单精度浮点数

对应C语言中的float型。



□ 64位双精度浮点数

对应C语言中的double型。



其中：

- 符号：取值0表示正数；取值1表示负数。
- 阶码：定点整数，用移码表示。
- 尾数：定点小数，用原码表示。

移码定义

假设真值 x 为定点整数， n 为 x 的移码表示中数值位的位数（比特数量）

$$[x]_{\text{移}} = x + 2^n \quad -2^n \leq x < 2^n$$

$$[x]_{\text{移}} = x + 2^7 \quad -2^7 \leq x < 2^7$$

考点四：数据表示——IEEE 754

■ 在IEEE 754 浮点数标准中，32位单精度浮点数的8位阶码尽管采用移码表示，但采用偏移常数是 $2^7 - 1 = 127$ ，而不是标准移码的 $2^7 = 128$ 。

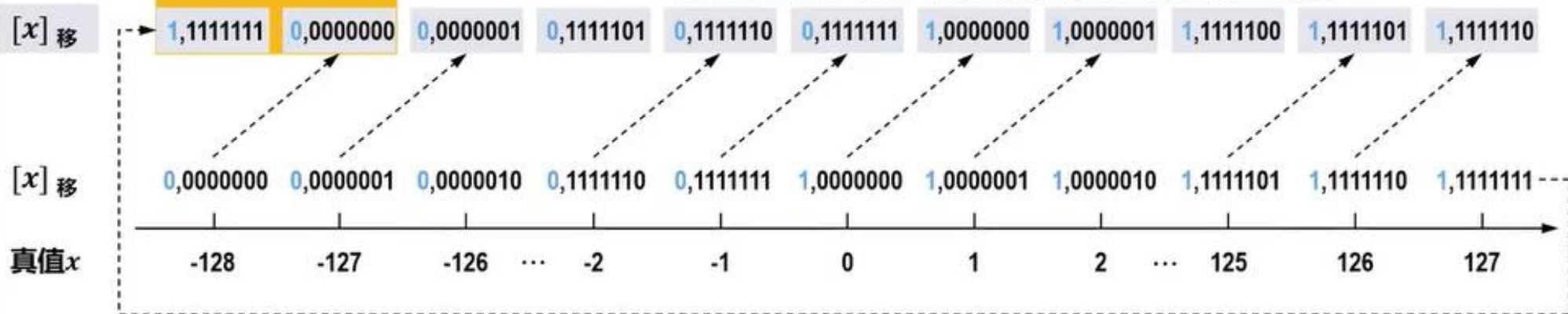
$$[x]_{\text{移}} = x + (2^7 - 1) \quad -2^7 \leq x < 2^7$$

为什么偏移常数不采用标准的128，而采用127？

采用偏移常数128表示的最小规格化数的倒数会发生溢出，而采用偏移常数127表示的任何一个规格化数的倒数则不会溢出。



阶码为全1、全0
有特殊用途



考点四：数据表示——IEEE 754

下面以32位单精度浮点数为例介绍IEEE 754单精度浮点数标准。



- 符号：取值0表示正数；取值1表示负数。
- 阶码：定点整数，用移码表示，偏置常数 $2^7 - 1 = 127$ 。
- 尾数：定点小数，用原码表示。符号位前移到最左侧。
相邻左侧隐藏一个1，表示数值而不表示符号。
尾数实际有24位，但不保存隐藏的那个1，只保存23位，节省的比特位可用于提高尾数的精度。
完整的尾数形式为1. M

考点四：数据表示——IEEE 754



假设 x 为阶码 E 的真值，即 $E = [x]_{\text{移}}$

$$[x]_{\text{移}} = e + 127$$

$$E = x + 127 \longrightarrow x = E - 127$$

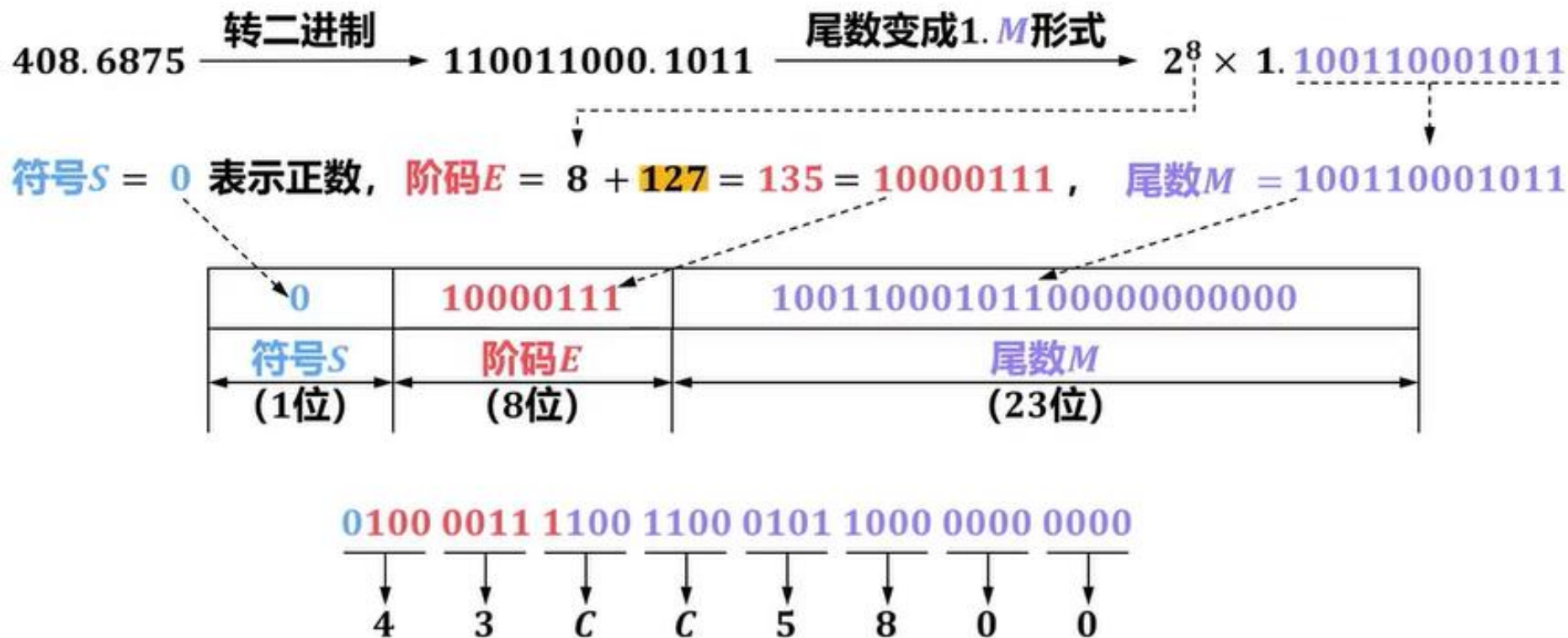


IEEE 754 单精度 (32位) 浮点数标准

数值的分类	符号 S	阶码 E	尾数 M	真值
正零	0	0 (全0)	0	$+0$
负零	1	0 (全0)	0	-0
正无穷大	0	255 (全1)	0	$+\infty$
负无穷大	1	255 (全1)	0	$-\infty$
无定义数 (非数)	0或1	255 (全1)	$\neq 0$	$NaN(not\ a\ number)$
规格化正数	0	$1 \leq E \leq 254$	M	$(-1)^0 \times 2^{E-127} \times 1.M$
规格化负数	1	$1 \leq E \leq 254$	M	$(-1)^1 \times 2^{E-127} \times 1.M$
非规格化正数	0	0 (全0)	$M \neq 0$	$(-1)^0 \times 2^{-126} \times 0.M$
非规格化负数	1	0 (全0)	$M \neq 0$	$(-1)^1 \times 2^{-126} \times 0.M$

考点四：数据表示——IEEE 754

【例】将十进制数408.6875转为IEEE 754单精度浮点的十六进制机器码？



考点四：数据表示——IEEE 754范围

数值的分类	符号 S	阶码 E	尾数 M	真值
规格化正数	0	$1 \leq E \leq 254$	M	$(-1)^0 \times 2^{E-127} \times 1.M$
规格化负数	1	$1 \leq E \leq 254$	M	$(-1)^1 \times 2^{E-127} \times 1.M$
非规格化正数	0	0 (全0)	$M \neq 0$	$(-1)^0 \times 2^{-126} \times 0.M$
非规格化负数	1	0 (全0)	$M \neq 0$	$(-1)^1 \times 2^{-126} \times 0.M$

$$1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + \dots + 1 \times 2^{-23} \quad \text{等比数列求和}$$

规格化正数最大值

$$(-1)^0 \times 2^{254-127} \times \boxed{1.111 \dots 11} = 2^{254-127} \times \boxed{(2 - 2^{-23})} \approx 3.4 \times 10^{38}$$

规格化正数最小值

$$(-1)^0 \times 2^{1-127} \times 1.000 \dots 00 = 2^{-126}$$

规格化负数最大值

$$(-1)^1 \times 2^{1-127} \times 1.000 \dots 00 = -2^{-126}$$

规格化负数最小值

$$(-1)^1 \times 2^{254-127} \times 1.111 \dots 11 = -(2^{254-127} \times (2 - 2^{-23})) \approx -3.4 \times 10^{38}$$